Low-Disruption Resilient Repair of Satellite Task Plans under Dynamic Disturbances

Anonymous Author(s)

Abstract

Plan-aware local repair addresses the need for fast, low-disruption recovery after part of a satellite task plan has entered the operational coordination chain. Existing methods mainly focus on static planning from scratch, global reoptimization after a disturbance, or stability-first minimal repair. They often omit the joint effects of issued commitments, disturbance locality, and insertion scale, which can lead to extensive movement of retained tasks or failure to recover available utility. Predictive–reactive scheduling and freeze-horizon rescheduling provide a basis for repair under an issued-prefix constraint. We therefore propose plan-aware local repair (PALR), a repair rule that treats issued commitments and disturbance locality as hard constraints. PALR combines frozen-prefix preservation, disturbance-centered neighborhood editing, an explicit insertion budget B, and lexicographic candidate selection to control the repair scope and the trade-off between utility recovery and plan stability. Comparative and ablation experiments under a unified protocol show that PALR reduces repair disruption while retaining utility comparable to greedy global reoptimization. In the cancellation setting, PALR reduces the inserted-task ratio by 37.8% and repair latency from 0.413 ms to 0.084 ms relative to greedy global reoptimization.

CCS Concepts

• **Computing methodologies** → **Planning and scheduling**.

Keywords

Earth-observation satellite scheduling, issued-prefix repair, low-disruption rescheduling, plan-aware local repair, online plan recovery

ACM Reference Format:

Anonymous Author(s). 2026. Low-Disruption Resilient Repair of Satellite Task Plans under Dynamic Disturbances. In . ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Earth-observation satellite scheduling has achieved substantial progress in observation selection, visibility-window matching, maneuver-constraint handling, and multi-satellite coordination. Although existing scheduling methods perform well in static plan construction, emergency-task insertion, and global re-optimization, they still struggle to jointly preserve an issued plan, respond locally to disturbances, and repair the plan efficiently. Once a plan enters the operational coordination chain, the scheduler no longer faces a pure utility-maximization problem, but rather a repair problem subject to existing commitments.

Conference'17, Washington, DC, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Existing studies advance Earth-observation scheduling from several perspectives. Based on prior results, these methods can be grouped into three categories. The first comprises static planners that construct high-quality plans from scratch through search, learning, and mathematical programming to optimize utility in static scenarios; however, they do not answer how a partially issued plan should be repaired. The second comprises global reoptimization methods after disturbances. Even when a disturbance is local, these methods may move many retained tasks and reopen decisions that operators expect to remain unchanged, and thus lack explicit protection for issued commitments. The third comprises stability-first local repair methods. If implemented too rigidly, they discard recoverable utility and cannot absorb locally feasible urgent tasks. PALR follows this third direction by treating plan stability as an explicit constraint while allowing bounded utility recovery.

In disturbance repair, stronger global search expands the editable region and increases repair latency, whereas overly conservative stability-first repair may leave newly released idle resources unused. Neither extreme jointly represents issued commitments, disturbance locality, and controllable insertion scale, although all three are relevant when plan preservation, local response, and low latency must be achieved simultaneously.

To avoid this dilemma, we model dynamic Earth-observation scheduling as locally controllable repair under an issued-prefix constraint rather than as global replanning. Hard preservation of the frozen prefix, disturbance-centered local editing, and an explicit insertion budget jointly balance utility recovery against disruption control. By limiting the editable region and explicitly accounting for every change, the resulting repair problem can, in principle, produce executable and low-disruption repairs. Fig. 1 illustrates the differences between global re-optimization and PALR local repair in edit scope and commitment preservation.

Operational repair is time-constrained: the system must progress from disturbance parsing to a feasible plan while handling cancellations, urgent insertions, and reward updates within computational and response-latency limits.

To address these issues, we draw on predictive–reactive scheduling and freeze-horizon rescheduling: first freeze part of the plan, then restrict editing to a local region, and finally use stability to balance utility. The principle of committing first and repairing locally afterward is consistent with our objective of low-disruption repair. Inspired by this principle, we construct the PALR repair rule and constrain it through neighborhood pruning and budget control. The frozen-prefix mechanism marks the issued prefix as immutable. Disturbance-centered neighborhood editing limits the propagation of changes, while the neighborhood width defines the editable region and insertion budget B limits the number of newly admitted tasks.

Unlike methods that treat the post-disturbance state as a new planning instance, PALR repairs the editable suffix while leaving

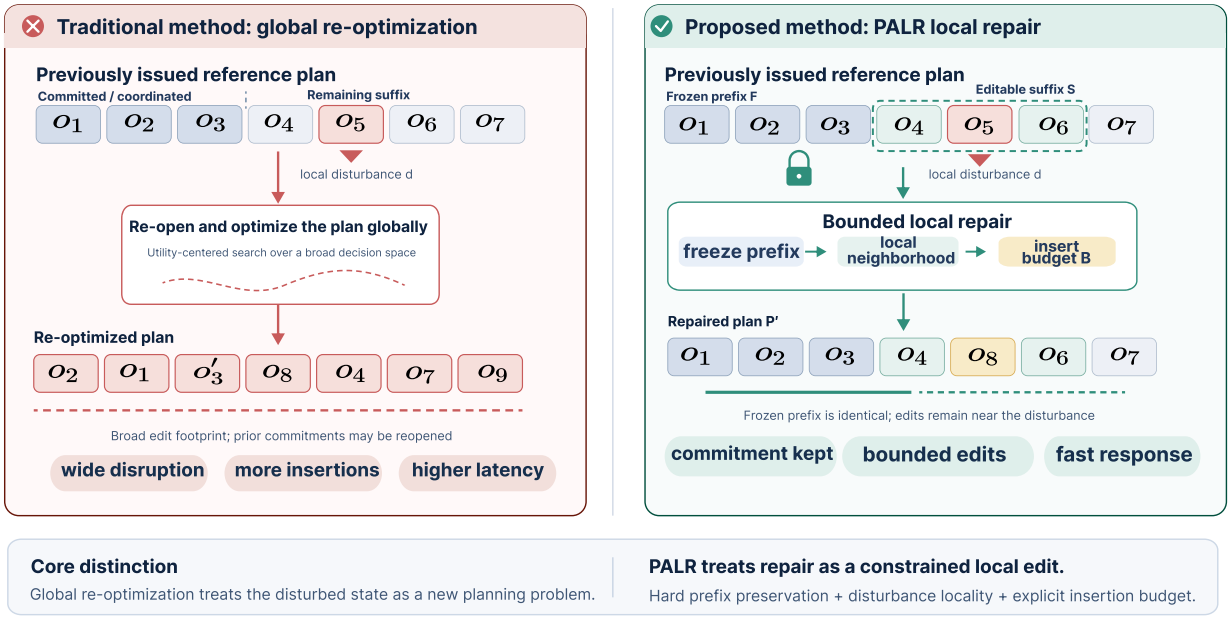


Figure 1: Conceptual comparison between global re-optimization and PALR local repair. Global re-optimization reopens coordinated decisions and produces a broad edit footprint, whereas PALR preserves the issued prefix and performs insertion-budget-constrained local edits only within the disturbance neighborhood.

the issued prefix unchanged. Frozen-prefix validity and insertion budget are explicit, auditable quantities rather than stability measures computed only after optimization. In summary, this study makes the following contributions: (1) we formulate dynamic Earth-observation scheduling as an issued-prefix repair problem, so that frozen-prefix preservation, utility recovery, and disruption control are evaluated jointly rather than measured only by a static objective value; (2) we propose the PALR repair rule, which unifies explicit frozen-prefix handling, a bounded editable neighborhood, and insertion budget B as an interpretable control variable within an auditable repair path; and (3) under a unified Earth-observation scheduling protocol, we provide variance-aware experimental evidence and characterize the operating range of bounded local repair, together with its observed degradation and escalation boundaries, through multi-metric comparisons with greedy and search-based replanning baselines.

The remainder of this paper is organized as follows. Section 2 positions this work in the context of Earth-observation planning and rescheduling repair. Section 3 defines the system model, protocol boundary, formal repair problem, and design requirements. Section 4 presents the PALR rule and its formal properties. Section 5 reports the experimental evidence.

2 Related Work

2.1 Earth-Observation Planning and Predictive–Reactive Rescheduling

Earth-observation satellite scheduling has advanced substantially in recent years, improving plan quality and constraint-handling capabilities. Distributed constraint optimization, online emergency scheduling, graph-guided search, and constraint programming have further extended the static utility frontier of Earth-observation scheduling [5, 11, 12, 18, 19, 29]. Planners that construct schedules from scratch obtain high-quality static plans by applying search, learning, and mathematical programming to candidate observations [11, 18]. Building on static planning, more complex frameworks, including distributed task planning, periodic onboard rescheduling, variable-scale deep reinforcement-learning schedulers, and bilevel optimization models, further extend dynamic response capabilities [8, 21–23, 26]. Beyond Earth observation, manufacturing and production research treats rescheduling as a trade-off between efficiency and stability. Reviews of multi-objective rescheduling, deep-reinforcement-learning-based dynamic scheduling, and digital-twin-driven scheduling provide adjacent methodological background for accountable repair of published plans [16, 28, 37]. However, the aforementioned Earth-observation methods mainly rely on construction from scratch or global re-optimization. They lack explicit protection for published commitments and do not model domain-specific structures such as visibility windows, maneuver feasibility, and frozen-prefix auditing. Bounded-disruption models and repair-cost trade-off methods alleviate this accountability gap by limiting the magnitude of changes in each rescheduling operation [9, 20]. By treating stability as a first-class constraint or trade-off objective, their attention to repair footprints motivates repair under an issued-prefix constraint and our elevation of the frozen prefix and insertion budget to first-class hard constraints.

2.2 Learning-Guided Destroy and Repair

The rise of learning-guided large-neighborhood search has promoted adaptive local repair, with the aim of producing higher-quality and more transferable repairs for disturbed combinatorial-optimization instances [2, 15, 17, 30, 31, 35, 39]. Large-neighborhood search has also been applied to classical scheduling problems such as bus-driver scheduling [27]. Learning-based acceleration for mixed-integer programming, including learned backdoor variables, and large-neighborhood ideas in multi-agent reinforcement learning extend this family from different perspectives [4, 10]. However, open-ended construction and improvement create challenges for published-plan repair. Under commitment-preservation constraints, learning-guided local repair provides a practical solution. Transferable local policies, memory-enhanced reinforcement frameworks, and cross-problem transfer methods balance search quality against transfer requirements through learned neighborhood control [13, 14, 24, 34, 38]. Global partitioning with local construction for large-scale routing, diffusion-based neural solvers, and local search over abstract constraint specifications further enrich solver design [1, 32, 36]. For satellite scheduling, a local-attention neural replanner improves adaptation in single-satellite agile optical scheduling through localized modeling [25]. The broader context of graph-based combinatorial optimization and search control is covered by related surveys and tree-search practice [3, 6, 7]. Both lines of work reflect a trend toward adaptive local repair for high-quality post-disturbance recovery. Although learned neighborhood control emphasizes search quality and transfer, while satellite neural replanners emphasize domain-specific structure, both overlook frozen-prefix semantics and auditable disruption trade-offs. Our method addresses these limitations by organizing frozen-prefix preservation, local-neighborhood editing, and an explicit insertion budget into an auditable repair process, thereby systematically addressing the difficulty of jointly achieving commitment preservation and low-disruption response in published-plan repair.

3 System and Problem Modeling

3.1 System Overview

We follow the established formulation of Earth-observation scheduling: an agile satellite must schedule an oversubscribed set of candidate observations, each with a visibility window, a utility value, and feasibility constraints induced by maneuvering and temporal behavior [18]. A plan is a time-ordered sequence of selected observations. In a dynamic setting, the scheduler receives both a task set and an existing plan whose prefix may already have been issued to operators and acted upon.

Let $P = (o_1, \dots, o_m)$ be the reference plan before a disturbance, and let k be the first editable position after issuance. The prefix $F = (o_1, \dots, o_{k-1})$ is frozen, whereas the suffix $(o_k, \dots, o_m)$ remains editable. A disturbance descriptor d specifies the change to be absorbed: a scheduled task may be canceled, a new urgent task may arrive, or utilities may be updated. The repair problem must produce a new feasible plan P' that leaves F unchanged, satisfies the Earth-observation constraints, and achieves a defensible trade-off between utility and disruption.

The repair workflow has two stages. First, an initial planner generates a reference plan for the current task set. Second, once part of that plan is considered issued and a disturbance occurs, the repair module is invoked. The module must account for committed decisions, determine how much of the remaining suffix must be unlocked, generate local candidate edits, and return a new feasible suffix quickly enough for online use. This workflow differs fundamentally from ordinary static replanning: the planning state now contains previously committed decisions, the disturbance type, and the remaining degrees of freedom. The repair algorithm must therefore track plan semantics rather than only objective values. A method that ignores the frozen prefix may appear strong under naive utility accounting, yet be operationally unacceptable because it invalidates already coordinated work.

3.2 Formal Problem Definition

3.2.1 Task and Plan Representation. An *observation task* is a tuple $o = (w_{\text{start}}, w_{\text{end}}, u, \theta)$, where $[w_{\text{start}}, w_{\text{end}}]$ is the visibility window in which the satellite can image the target, $u \in \mathbb{R}^+$ is its utility, and θ encodes geometric and operational constraints, such as roll-angle limits and downlink requirements. Let $\mathcal{O}$ denote the candidate task set. A *plan* $P = (o_1, o_2, \dots, o_m)$ is a time-ordered sequence of selected observations, where $o_i \in \mathcal{O}$ and each task appears at most once. The *utility* of the plan is $U(P) = \sum_{i=1}^{m} u(o_i)$.

3.2.2 Feasibility Constraints. A plan P is *feasible* if it satisfies the following conditions:

- **Visibility-window constraint:** Each observation o_i must be scheduled within its window. If t_i is its start time in P, then
$$t_i \in [w_{\text{start}}(o_i), w_{\text{end}}(o_i)].$$
- **Maneuver-separation constraint:** Adjacent observations must leave sufficient time for the satellite to roll between targets. Formally, let $\Delta(o_i, o_{i+1})$ be the minimum roll time between o_i and o_{i+1}, computed from their angular separation and the satellite's maximum roll rate. We require $t_{i+1} - (t_i + \text{dur}(o_i)) \geq \Delta(o_i, o_{i+1})$, where $\text{dur}(o_i)$ is the imaging duration of o_i.
- **No-duplication constraint:** Each observation appears at most once in P.

We write $\text{feasible}(P) = 1$ if all constraints hold, and $\text{feasible}(P) = 0$ otherwise.

3.2.3 Repair under an Issued-Prefix Constraint. Let $P = (o_1, \dots, o_m)$ be the *reference plan* before a disturbance. After part of the plan has been issued, the first $k-1$ observations are committed and immutable. We call $F = (o_1, \dots, o_{k-1})$ the *frozen prefix* (or *issued prefix*) and $S = (o_k, \dots, o_m)$ the *editable suffix*. A *disturbance* d specifies a change to the task set or utilities:

- **Cancellation:** A scheduled task $o_c \in S$ becomes unavailable and must be removed.
- **Urgent insertion:** A new high-priority task $o_u \notin P$ arrives and should be accommodated when feasible.
- **Reward update:** The utilities of some tasks change, requiring the plan to be adjusted to capture the new value.

The *repair problem* seeks a new plan P' such that

$$\max_{P'} U(P') \tag{1}$$

subject to:

$$P'[1 : k-1] = F \quad \text{(frozen-prefix preservation)} \tag{2}$$

$$\text{feasible}(P') = 1 \quad \text{(Earth-observation constraints)} \tag{3}$$

Among plans with comparable utility, repair further prefers smaller *task-set changes* (the number of added or removed observations), a lower *insertion burden* (the number of new tasks), and a smaller *time shift* (the total scheduled movement of retained tasks). Utility and disruption are therefore part of the problem definition rather than post hoc diagnostics.

3.2.4 Locality and the Disturbance-Centered Neighborhood. A key design choice in PALR is to restrict editing to a *disturbance-centered neighborhood*. Let $\text{pos}(d)$ denote the plan position most affected by disturbance d, such as the index of a canceled task or the preferred insertion point of an urgent task. The *width-w neighborhood* around d is defined as

$$N_w(d) = \{o_i \in S : |\text{index}(o_i) - \text{pos}(d)| \le w\} \tag{4}$$

PALR restricts local search to $N_w(d)$, thereby imposing a spatial bound on repair-induced disruption. Observations outside $N_w(d)$ remain unchanged in the repaired plan.

3.2.5 Insertion-Budget Control. To further limit repair intensity, PALR introduces an explicit *insertion budget* $B \in \mathbb{Z}_{\ge 0}$ that bounds the number of tasks newly added during repair:

$$|P'_{\text{new}}| \le B \tag{5}$$

where $P'_{\text{new}} = P' \setminus P$ is the set of newly inserted observations. The parameter B is an interpretable control knob: a smaller B prioritizes plan stability, whereas a larger B permits more aggressive utility recovery.

3.3 Evaluation Protocol and Complexity

Because the private assets of the closest Earth-observation graph-neural search planner are unavailable, we evaluate all methods under a unified protocol adopted in this study and do not claim an exact reproduction [18]. The protocol preserves the task formulation, frozen-prefix semantics, and repair metrics of interest; all absolute values are therefore interpreted only as within-protocol comparisons. We use $\text{FR} \in [0, 1]$ to denote the frozen-prefix ratio: $\text{FR} = 0$ means that no prefix has been issued, whereas $\text{FR} = 0.6$ means that 60% of the reference plan has been committed before repair begins. All methods run on the same instance generator with matched random seeds and the same repair-oriented metric set. Latency is thus interpreted as relative repair cost along a shared evaluation path rather than end-to-end mission-system latency.

The exact repair problem is NP-hard in its natural decision form. When $\text{FR} = 0$ and no local-edit restriction is imposed, it reduces to static Earth-observation scheduling: selecting and ordering a utility-maximizing subset subject to visibility windows and sequence-dependent maneuver constraints. This base problem is widely recognized as difficult in the Earth-observation scheduling literature [5, 12, 18]. An issued prefix and bounded repair actions add combinatorial structure rather than simplifying the problem.

3.4 Post-Issuance Failure Modes and Design Requirements

Earth-observation scheduling for an issued plan is not simply "static planning plus a disturbance." A disturbance propagates through a partially committed structure. Cancellation releases local idle capacity, urgent insertion creates tension between responsiveness and commitment preservation, and reward updates preserve plan topology while changing the utility ranking. These effects expose a limitation of utility-only comparisons: a repair method may report higher utility while relying on extensive movement of retained tasks, large insertion cascades, or violations of the issued prefix.

The preceding analysis yields five design requirements for a reliability-oriented repair system.

- **G1: Commitment preservation.** The frozen or issued prefix must be modeled explicitly; violations must be reported rather than hidden.
- **G2: Low-disruption adaptation.** The method should keep task-set changes, insertion counts, and time shifts of retained tasks small.
- **G3: Fast response.** The repair cycle should require substantially less computation than heavier global replanning or search.
- **G4: Disturbance coverage.** The repair method should handle cancellation, urgent insertion, and reward updates from a unified repair perspective.
- **G5: Auditable behavior.** Successes and failure modes should be explainable through explicit local edits and measurable operational consequences.

PALR operationalizes these requirements through explicit frozen-prefix validation, bounded local editing, insertion budget B, and repair metrics that report utility, disruption, and latency.

4 Method: The Plan-Aware Local Repair Rule

4.1 Section Overview

This section details the proposed plan-aware local repair rule (PALR rule). Given a reference plan, a disturbance descriptor, and a frozen-prefix boundary, the rule returns a repaired plan together with utility, disruption, and validity diagnostics, as shown in Fig. 2.

4.2 Overall Framework

Motivated by the facts that an issued plan cannot be modified arbitrarily and that disturbance response should remain local, and guided by the principle that both commitment preservation and insertion scale should be hard constraints, we propose the PALR rule centered on bounded local editing. First, to address the immutability of the issued prefix, we design a frozen-prefix preservation mechanism. Second, to prevent repair-induced movement from propagating to distant positions, we design a disturbance-centered neighborhood-editing mechanism. Third, an explicit insertion budget B controls the number of newly admitted tasks. Finally, the three mechanisms are connected in a single repair process, with prefix preservation and feasibility treated as hard constraints and utility and disruption as soft objectives. This choice makes PALR a repair method rather than a general-purpose replanner.

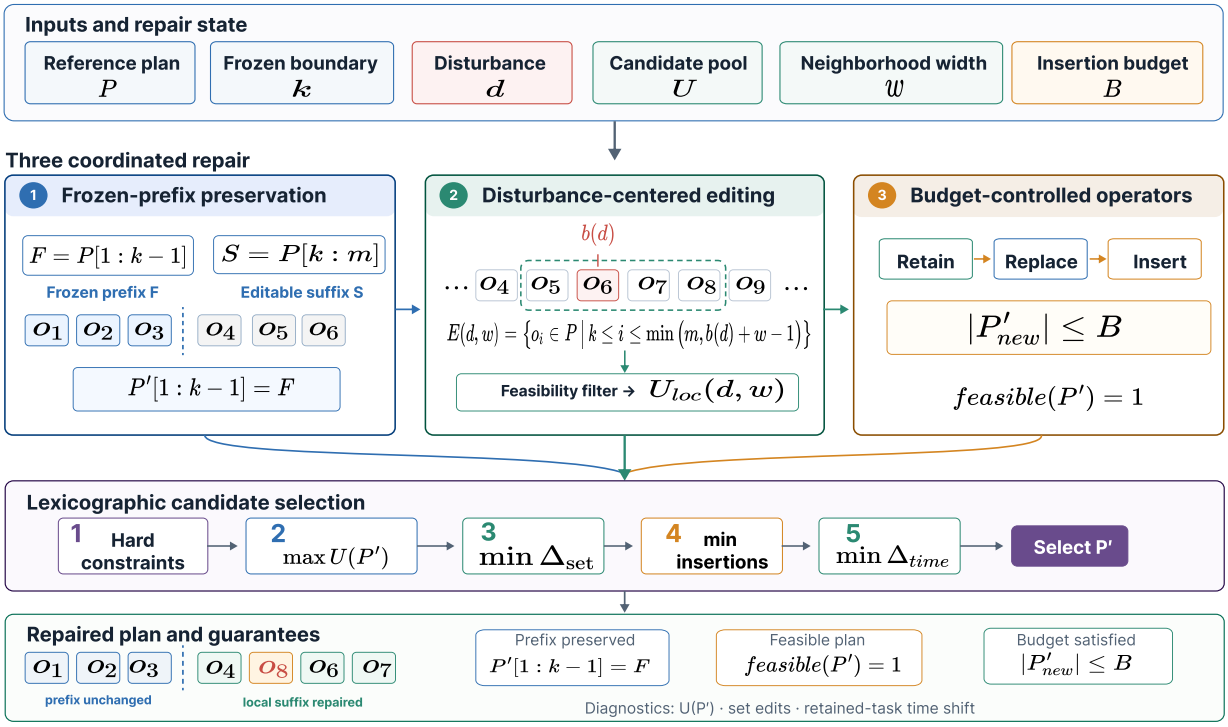


Figure 2: Overall PALR framework. The method takes a reference plan, frozen boundary, disturbance, candidate pool, neighborhood width, and insertion budget as inputs. Frozen-prefix preservation, disturbance-centered neighborhood editing, and budget-controlled operators generate locally feasible candidates, which are selected lexicographically according to feasibility, utility, and disruption metrics to produce a repaired plan that preserves the frozen prefix.

4.3 Detailed Design

Module 1 (frozen-prefix preservation): This mechanism locks the part of the plan that has already entered the operational coordination chain, preventing repair from touching these committed decisions. Specifically:

- Input specification: the reference plan $P = (o_1, \ldots, o_m)$ and the first editable position k, where the prefix $F = (o_1, \ldots, o_{k-1})$ is treated as frozen and the suffix $(o_k, \ldots, o_m)$ is editable;
- Formulation: the hard constraint requires

$$P'[1 : k-1] = F, \tag{6}$$

where P' is the repaired plan; that is, the first $k-1$ observations must remain position-wise identical to those in the original plan after repair;
- Output specification: a candidate plan that changes only at positions $i \geq k$ and satisfies the validity criterion of position-wise frozen-prefix preservation.

The output of this module (the locked prefix and open suffix) is passed as input to Modules 2 and 3.

Module 2 (disturbance-centered neighborhood editing): To confine repair-induced movement to the vicinity of the disturbance, we introduce two components, neighborhood delimitation and local candidate-pool filtering, so that only the necessary suffix is reopened while the remainder stays unchanged when the disturbance is local.

- Component 1 (neighborhood delimitation): an unlocked window centered on the disturbance boundary with width w is used because only positions within this window are allowed to change. Formally, it is defined as

$$E(d, w) = \{o_i \in P \mid k \leq i \leq \min(m, b(d) + w - 1)\}, \tag{7}$$

where d is the disturbance descriptor, $b(d)$ is the first suffix position affected by disturbance d, and w is the neighborhood width;
- Component 2 (local candidate filtering): using the time span covered by $E(d, w)$ as the filtering condition, we require the local candidate pool $U_{\text{loc}}(d, w) \subseteq U$ to contain only unscheduled tasks whose visibility windows intersect either this span or the idle interval released by the disturbance. This allows the rule to first attempt to absorb the disturbance near its causal boundary before spending computation on distant parts of the suffix.

Component 1 (neighborhood delimitation) provides the basis for Component 2 (local candidate filtering), and the two are executed sequentially. This module receives the locked prefix from Module 1, and passes its outputs (the unlocked neighborhood and local candidate pool) to Module 3.

Module 3 (budget-controlled repair operators): The repair-operator space is organized around three actions—retain, replace, and insert—and comprises the following three sequential steps:

- Step 1 (retain): for each observation o_i in the neighborhood that has not been canceled and is compatible with the already constructed part, append it to P' at its original position. This step restores local feasibility without moving retained tasks;
- Step 2 (replace): when an observation o_i in the neighborhood can be replaced by a higher-utility observation o_j from the local candidate pool and the replacement remains feasible, perform the exchange. Formally, $u(o_j) > u(o_i)$ and $\text{feasible}(P') = 1$ are required, after which the candidate plan is updated accordingly;
- Step 3 (budgeted insertion): finally, while the insertion budget remains, greedily add locally feasible new tasks. Specifically, select $o_{\text{best}} = \arg\max_o\{u(o) : \text{feasible}(P' \cup \{o\}) = 1\}$ at each iteration and increment the insertion count until the budget limit is reached.

The three steps have a strict sequential relationship: each preceding step provides the basis for the next, and they must be executed in order. This module receives inputs from Modules 1 and 2, and its output is the final repaired plan of the system. The insertion step is limited by the hard constraint

$$|\{a_i \in \pi : a_i = \text{Insert}(\cdot)\}| \le B \tag{8}$$

where π is the sequence of operators applied within the neighborhood and B is the insertion budget.

4.4 Module Integration

To address the challenge of handling three disturbance types uniformly under a single rule, we propose an integration scheme that evaluates candidate plans using lexicographic preferences. Because utility recovery and disruption control are in tension, we distinguish hard constraints from soft objectives through the following decision order, which is then unfolded into a repair process: first reject any candidate that changes the frozen prefix or violates feasibility; then maximize recovered utility; and finally minimize, in sequence, task-set changes, the number of inserted tasks, and the temporal movement of retained tasks.

Algorithm 3 formalizes the above process. It takes the reference plan P, frozen boundary k, disturbance descriptor d, candidate pool U, neighborhood width w, and insertion budget B as inputs, and returns a repaired plan P' that preserves $F = P[1 : k-1]$ and satisfies Earth-observation feasibility.

In summary, this method can not only exploit newly released idle intervals to recover utility in cancellation scenarios, but also admit new high-value opportunities after reward updates without moving retained tasks. By combining prefix preservation, local neighborhoods, and insertion budgets into a single process, it achieves the central objective of using low-disruption local repair by default and escalating to heavier search only when necessary. Mathematically, the insertion budget B is the only control variable validated in this paper: at $B = 0$, the method degenerates to a strict stability-first baseline; increasing B enlarges the set of admissible insertions, thereby adjusting repair intensity between utility recovery and schedule stability.

```
Algorithm 1: PALR rule
Input: P (reference plan), k (frozen boundary),
       d (disturbance), U (candidate pool),
       w (neighborhood width), B (insertion budget)
Output: P' (repaired plan)

1: F <- P[1:k-1]  // frozen prefix
2: S <- P[k:m]    // editable suffix
3: E <- BuildNeighborhood(S, d, w)
4: U_loc <- FilterCandidates(U, E)
5: P' <- F         // initialize with frozen prefix
6:
7: // Step 1: minimal repair (restore feasibility)
8: for each o_i in E do
9:    if o_i != canceled task and Compatible(o_i, P') then
10:      P' <- P' union {o_i}  // retain
11:   end if
12: end for
13:
14: // Step 2: budget-controlled insertion
15: inserted <- 0
16: while inserted < B do
17:   o_best <- argmax_{o in U_loc\P'}
18:             {u(o) : Feasible(P' union {o})}
19:   if o_best exists then
20:     P' <- P' union {o_best}
21:     inserted <- inserted + 1
22:   else
23:     break  // no further feasible insertion
24:   end if
25: end while
26:
27: return P'
```

Figure 3: PALR repair rule. The algorithm preserves the frozen prefix F, restores feasibility within the local neighborhood E, and greedily inserts at most B new tasks.

4.5 Properties and Boundary Analysis

Proposition 1 (Prefix preservation). *Let $F = (o_1, \ldots, o_{k-1})$ be the frozen prefix of the input plan P. For every valid input, the repaired plan P' produced by the PALR rule satisfies $P'[1 : k-1] = F$.*

Proof sketch. The rule constructs candidates only within the editable suffix $i \ge k$. Any candidate that changes a frozen-prefix position is rejected before utility ranking, and the remaining candidates must also pass the feasibility check. Therefore, every admissible output preserves F. Although Proposition 1 is elementary, it is central to the reliability argument of this paper: a strict urgent-insertion success is counted only when the repaired schedule preserves the frozen prefix.

Theorem 1 (NP-hardness). *The decision version of the dynamic Earth-observation satellite repair problem is NP-hard.*

Proof sketch. When $F = \emptyset$ (no frozen prefix) and the budget is unlimited, the repair problem reduces to static Earth-observation satellite scheduling, which has been proved NP-hard [5, 18]. This

hardness result motivates bounded heuristic methods such as PALR, which trade optimality for speed and predictability.

Proposition 2 (Monotonicity of utility recovery)**.** *For a fixed input, increasing the insertion budget B never decreases the utility of the repaired plan: if $B_1 \le B_2$, then $U(P'_{B_1}) \le U(P'_{B_2})$.*

The proof follows from the greedy insertion strategy in Algorithm 3. However, monotonicity does not extend to the disruption metrics: higher budgets increase task-set changes and insertion burden, as observed in the experiments. The insertion budget B therefore defines a practical repair frontier—at $B = 0$, PALR degenerates to minimal repair, whereas larger budgets extend local recovery but may conflict with the original low-disruption objective.

These results also delimit the current claims. Under urgent insertion with FR = 0, PALR is numerically identical to minimal repair; therefore, the rule is not yet a richer urgent-placement mechanism. Its supported role is narrower: a repairer for cancellations and reward updates with an explicit budget frontier. Under urgent insertion with a heavily frozen prefix, learned-prior PUCT recovers more utility under the same hard frozen constraint, but at the cost of greater task-set disruption, greater movement of retained tasks, and approximately three orders of magnitude higher latency than PALR. Most prefix-preserving methods achieve a strict urgent-insertion success rate of 0.34 (17 of 50 instances); operator-policy repair is the sole exception, with a rate of 0.32 (16 of 50). These results support using local repair by default and escalating only when the additional utility justifies the higher latency and movement.

5 Experiments

5.1 Experimental Setup

All reported main results use a single accepted, paper-aligned protocol: 50 instances, 100 candidate observations per instance, a planning horizon of $\tau = 5400$, and fixed random seeds to ensure comparability. We evaluate three disturbance types: cancellation, urgent insertion, and reward update. Disturbances are applied after the reference plan is generated, and the frozen-prefix ratio (FR) controls the proportion of the plan that has already been committed before repair begins.

To evaluate the performance of the proposed PALR rule more comprehensively, we compare it with two broad classes of baselines: global reoptimization and stability-first repair. The global-reoptimization class is represented by greedy global reoptimization, a natural global-planning baseline for Earth-observation satellites [5, 11, 18, 19, 29, 33] that aggressively reopens the editable suffix. To strengthen the comparison with graph-guided tree-search planning, we report two inference-time PUCT comparators under the same frozen-aware repair framework: a heuristic PUCT replanner and a learned-prior variant. The latter uses 40 simulations, a maximum depth of 12, top-k candidate control of 20, an exploration constant of 1.5, and a shallow guidance depth of 2, drawing on the graph-neural tree-search line for Earth-observation satellites [18] and standard Monte Carlo tree-search practice [7]. The stability-first repair class is represented by minimal repair, following recent stability-first and bounded-disruption repair formulations [9, 16, 20, 28, 37]. Where robustness questions benefit from additional low-disruption comparisons, we further include a learned local-repair variant and an operator-policy repair variant [3]. The learned-prior row changes only the search prior, not the repair contract, and is therefore used as an informative comparator rather than a reproduction of a complete graph-neural tree-search training stack.

Table 1: Cancellation scenario (means ± 95% confidence intervals over 50 instances). Utility is in units of $\times 10^9$.

Method	Utility	Set change	Insertion	Movement	Latency
Greedy global reoptimization	2.348±0.259	0.048±0.028	0.539±0.422	24.6±12.1	0.413±0.041
Minimal repair	2.222±0.273	0.000	0.000	0.0	0.009±0.001
PALR rule ($B = 2$)	2.362±0.273	0.000	0.335±0.108	0.0	0.084±0.005

The experiments assess commitment-preserving repair quality using four groups of metrics. Utility measures the total observation value of the repaired plan and evaluates utility recovery. The disruption group (task-set changes, insertion burden, and temporal movement of retained tasks) measures plan changes introduced by repair and evaluates low-disruption capability. Latency measures the relative repair cost on the shared evaluation path and evaluates rapid-response capability. Strict validity measures the urgent-insertion success rate subject to frozen-prefix preservation and evaluates commitment-preservation capability. This grouping is deliberate: commitment-preserving repair should be judged on a joint surface rather than by utility alone. We report means ± 95% confidence intervals for selected comparisons between closely performing methods, computed using a t distribution with $n - 1 = 49$ degrees of freedom. We use paired t-tests for utility, set changes, insertion burden, and temporal movement; Welch's t-test for latency; and Bonferroni correction for multiple comparisons.

All methods are executed on the same instance generator, with matched random seeds and the same repair-oriented metric set. The protocol is aligned with the task boundary of [18]: the same issued-prefix repair problem, the same visibility-window and maneuver-feasibility structure, and the same repair-oriented metrics, but without reproducing that paper's private resources or absolute reward scale. Consequently, all reported values are interpreted only within this protocol, latency is read as relative repair cost, and reward-update utility is not compared across disturbance families.

5.2 Main Results

Cancellation. Table 1 reports the clearest positive result in this paper. With a budget of 2, the PALR rule slightly exceeds greedy global reoptimization in utility while maintaining zero task-set changes and zero temporal movement of retained tasks. An important qualification is that PALR should be understood as a middle path, rather than the minimum-disruption winner on every axis; when insertion burden is considered in isolation, minimal repair remains the most conservative baseline. Under the same cancellation protocol, PALR reaches the same utility range while reducing insertion burden from 0.539 to 0.335, a decrease of approximately 37.8%; eliminating retained-task movement; and reducing latency from 0.413 ms to 0.084 ms, a decrease of approximately 79.7%. For operators, this is precisely the practically meaningful region in which a low-disruption repairer improves upon aggressive suffix reconstruction.

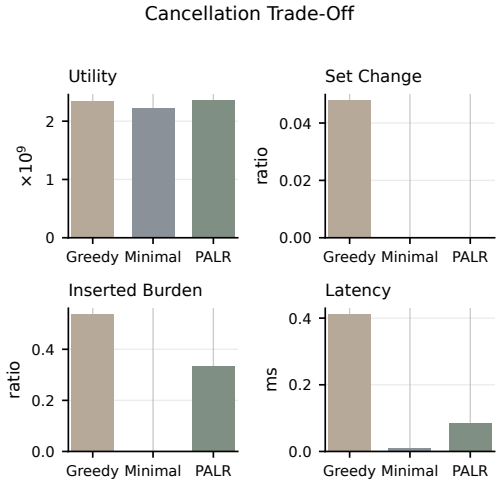


Figure 4: Trade-offs in utility, set changes, insertion burden, and latency under cancellation. The PALR rule occupies the middle region of the utility–disruption frontier: minimal repair is the zero-insertion endpoint, whereas greedy global reoptimization is the aggressive-reinsertion endpoint.

Table 2: Urgent insertion without a frozen prefix (means ± 95% confidence intervals; FR = 0.0).

Method	Utility	Set change	Insertion	Movement
Greedy global reoptimization	2.417±0.276	0.142±0.061	0.168±0.041	34.4±21.6
Minimal repair	2.937±0.220	0.009±0.010	0.168±0.041	11.7±10.7
PALR rule	2.937±0.220	0.009±0.010	0.168±0.041	11.7±10.7

The cancellation result is best interpreted as a trade-off surface rather than a single-score victory. Minimal repair is the zero-insertion endpoint, greedy global reoptimization is the aggressive-reinsertion endpoint, and PALR occupies the middle region by accepting some insertions to recover utility while remaining below greedy global reoptimization in insertion volume and latency. The small utility difference between PALR and greedy global reoptimization is therefore less informative than their differences in disruption and latency.

Urgent insertion. Tables 2 and 3 characterize the urgent-insertion regime without a frozen prefix. Under this setting, PALR degenerates to minimal repair: the two rows are numerically identical in utility and disruption, and the slight latency difference is attributable to implementation overhead. This regime supports the issued-prefix repair framing more strongly than a distinct urgent-placement contribution. All three methods place the urgent task, but minimal repair and PALR do so with less schedule movement than greedy global reoptimization. When FR = 0 and the urgent task can already be admitted through a single minimal local edit, PALR has no additional ranking signal that would justify broader neighborhood expansion. We therefore treat this regime as a boundary case rather than a second headline result.

Table 3: Latency and strict success rate for urgent insertion (means ± 95% confidence intervals where applicable; FR = 0.0).

Method	Latency (ms)	Strict success rate
Greedy global reoptimization	0.406±0.044	1.00
Minimal repair	0.038±0.009	1.00
PALR rule	0.039±0.013	1.00

Table 4: Cancellation ablation (means ± 95% confidence intervals). Greedy global reoptimization is included as a reference row.

Budget	Utility (×10^9)	Inserted-task ratio	Latency (ms)
Greedy global reoptimization (reference)	2.348±0.259	0.539±0.422	0.413±0.041
$B = 0$	2.222±0.273	0.000	0.012±0.001
$B = 1$	2.313±0.271	0.202±0.056	0.054±0.004
$B = 2$	2.362±0.273	0.335±0.108	0.087±0.007
$B = 3$	2.375±0.274	0.429±0.145	0.106±0.010
$B = 5$	2.388±0.276	0.585±0.229	0.130±0.018

Reward update. Reward updates expose a third repair mode: the current PALR rule acts as an insertion-only repairer, keeping both task-set disruption and retained-task movement at zero and recovering utility only through additional insertions. Because the disturbance itself changes the reward scale, the absolute utility values in Tables 8 and 9 should be compared only within their respective tables. This distinction matters in practice because reward updates are common in rolling planning even when the geometry remains unchanged. A repairer that can enrich the editable tail without touching retained tasks gives operators a way to respond to new value estimates while keeping prior coordination stable. The cost is that such a repairer performs poorly when the new value landscape requires deeper structural rearrangement, which is why we treat reward updates as a complementary rather than universal success case.

5.3 Ablation Study

To quantify how the insertion budget controls PALR's behavior, we conduct a budget-sweep ablation in the cancellation regime, with results summarized in Table 4. As B increases, utility improves, but insertion volume and latency also increase. A budget of 2 is the best-supported default in this sweep because it matches or slightly exceeds greedy global reoptimization in utility while using fewer insertions and less time.

The budget sweep shows that a positive insertion budget shifts PALR from pure preservation toward controlled recovery. Across the evaluated values of B, utility increases monotonically together with insertion burden, supporting the use of B as an interpretable control variable. Moving from $B = 0$ to $B = 2$ yields most of the observed utility recovery, whereas moving from $B = 2$ to $B = 5$ produces smaller utility gains with continued growth in inserted-task burden. We therefore use $B = 2$ as the default operating point in this protocol. The ablation shows that B directly controls the amount of insertion capacity spent on utility recovery while keeping the repair scope explicit and auditable.

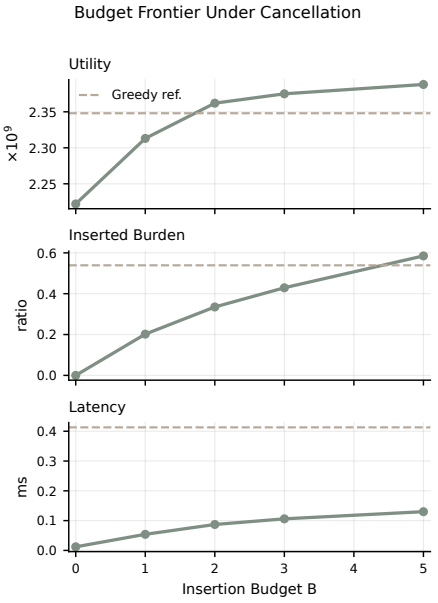


Figure 5: Cancellation budget frontier. The best-supported operating point is $B = 2$: larger budgets still yield slight utility gains, but the accompanying increase in insertion burden erodes the low-disruption advantage.

5.4 Robustness Evaluation

The robustness results clarify when PALR should and should not be trusted. We focus on the regimes that most clearly stress commitment preservation: heavily frozen prefixes, urgent insertion under strict validity, and non-topological reward changes.

5.4.1 Cancellation under a Heavily Frozen Prefix. Table 5 shows that at a frozen ratio of 0.6, PALR remains close to the utility of greedy global reoptimization while maintaining zero task-set disruption and essentially zero retained-task movement. This is a favorable case for bounded repair: the editable tail is small, yet PALR recovers most of the available value without reopening retained work. This regime matters because the editable tail is already short before the disturbance occurs, making utility recovery without moving retained work more difficult than in the basic cancellation setting. PALR remains near the utility band of greedy global reoptimization at both intensities, supporting the narrower claim that bounded local repair remains viable when a disturbance can be absorbed through tail-level rearrangement.

5.4.2 Urgent Insertion under a Heavily Frozen Prefix. The heavily frozen urgent regime is where the strongest utility-oriented comparator matters most. Tables 6 and 7 show that the learned-prior PUCT comparator achieves the highest utility under the hard frozen constraint, but incurs substantially greater latency and movement than the local-repair family. Most prefix-preserving methods achieve a strict urgent-success rate of 0.34 (17 of 50 instances); operator-policy repair is the sole exception, with a rate of 0.32 (16 of 50). Because the observed success rate reflects both instance feasibility and algorithm behavior, these results do not establish that

Table 5: Cancellation robustness under a heavily frozen constraint (means ± 95% confidence intervals). Utility is in units of $\times 10^9$.

Setting	Method	Utility	Set change	Insertion	Movement
FR = 0.6, intensity = 0.5					
	Greedy global reoptimization	2.413±0.270	0.056±0.028	0.597±0.417	12.5±8.3
	Minimal repair	2.298±0.289	0.000	0.000	0.0
	PALR rule ($B = 2$)	2.398±0.275	0.000	0.313±0.110	0.0
FR = 0.6, intensity = 1.0					
	Greedy global reoptimization	2.373±0.258	0.053±0.029	0.564±0.419	20.0±9.3
	Minimal repair	2.256±0.277	0.000	0.000	0.021±0.042
	PALR rule ($B = 2$)	2.364±0.268	0.000	0.342±0.106	0.021±0.042

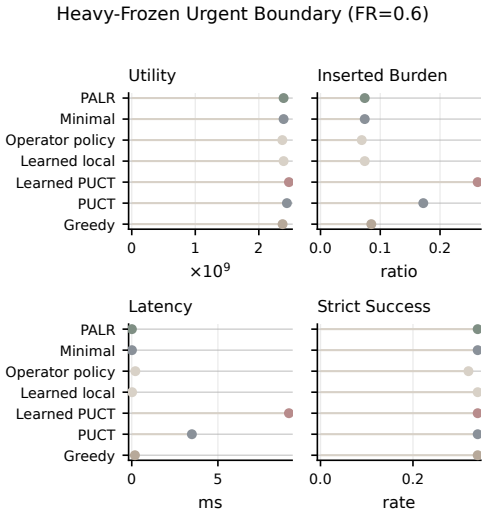


Figure 6: Heavily frozen urgent boundary at FR = 0.6. Stronger search improves urgent utility while matching the maximum observed strict-success rate of 0.34, but incurs substantially greater latency and a larger inserted-task burden than local repair.

Table 6: Urgent insertion under heavy freezing (means ± 95% confidence intervals; FR = 0.6). The two PUCT rows share the same budget of 40 simulations, while the PALR rule and minimal repair share the same urgent-repair decision in the current implementation.

Method	Utility	Set change	Insertion	Movement
Greedy global reoptimization	2.374±0.220	0.041±0.027	0.085±0.049	2.4±1.6
PUCT reoptimization	2.442±0.220	0.041±0.027	0.172±0.127	2.8±1.7
Learned-prior PUCT	2.474±0.224	0.056±0.026	0.263±0.158	9.5±3.1
Learned local repair	2.390±0.223	0.028±0.022	0.074±0.045	0.4±0.5
Operator-policy repair	2.370±0.223	0.034±0.023	0.069±0.045	0.6±0.6
Minimal repair	2.390±0.223	0.028±0.022	0.074±0.045	1.9±1.5
PALR rule	2.390±0.223	0.028±0.022	0.074±0.045	1.9±1.5

the remaining instances are infeasible. Stronger search therefore improves the quality of successful repairs without increasing the highest observed strict-success rate.

The same tables also show why learned-prior PUCT is not an unconditional overall winner: its additional utility is accompanied by a larger inserted-task burden and a latency penalty relative to

Table 7: Urgent insertion under heavy freezing: latency and strict success rate (means ± 95% confidence intervals where applicable; FR = 0.6).

Method	Latency (ms)	Strict success rate
Greedy global reoptimization	0.184±0.013	0.34
PUCT reoptimization	3.489±0.440	0.34
Learned-prior PUCT	9.122±1.813	0.34
Learned local repair	0.021±0.009	0.34
Operator-policy repair	0.209±0.020	0.32
Minimal repair	0.011±0.003	0.34
PALR rule	0.008±0.002	0.34

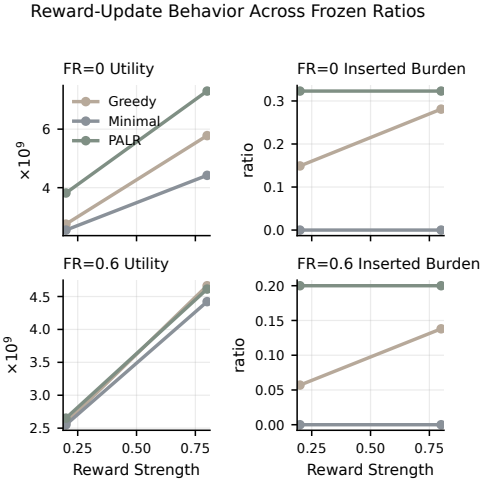


Figure 7: Reward-update behavior across frozen ratios. The PALR rule behaves as an insertion-only repairer: it preserves retained work, but its utility advantage narrows sharply once the editable tail becomes smaller.

low-disruption repair. Under this protocol, most methods achieve 17 strict successes out of 50, whereas operator-policy repair achieves 16. The utility comparison should therefore be interpreted together with the differences in latency, insertion burden, and movement.

5.4.3 Reward Updates as Non-Topological Disturbances. Tables 8 and 9 report a reward-update grid across two frozen ratios and two disturbance intensities. The key observation is that the current PALR rule behaves as an insertion-only repairer: it leaves the original retained plan unchanged and gains utility by appending or activating new opportunities in the editable tail. In contrast, greedy global reoptimization gains utility by reopening the suffix more aggressively, which increases movement and task-set changes.

The reward-update tables are best read as two distinct repair modes. At FR = 0, the PALR rule can spend a small number of insertions to obtain large utility gains without moving retained work. At FR = 0.6, the same strategy is compressed into a much thinner editable tail; preservation still holds, but most of the room for value recovery disappears. Reward updates therefore distinguish

Table 8: Reward updates without a frozen prefix (means ± 95% confidence intervals; FR = 0.0). Utility is in units of $\times 10^9$ and is comparable only within this disturbance family.

Intensity	Method	Utility	Set	Added	Movement
0.2	Greedy global reoptimization	2.755±0.308	0.132±0.049	0.149±0.041	36.5±15.8
0.2	Minimal repair	2.552±0.298	0.000	0.000	0.0
0.2	PALR rule	3.818±0.321	0.000	0.323±0.053	0.0
0.8	Greedy global reoptimization	5.779±0.784	0.239±0.052	0.281±0.071	61.8±17.5
0.8	Minimal repair	4.422±0.701	0.000	0.000	0.0
0.8	PALR rule	7.301±0.767	0.000	0.323±0.053	0.0

Table 9: Reward updates under a heavily frozen prefix (means ± 95% confidence intervals; FR = 0.6). Utility is in units of $\times 10^9$ and is comparable only within this disturbance family.

Intensity	Method	Utility	Set	Added	Movement
0.2	Greedy global reoptimization	2.583±0.298	0.041±0.032	0.057±0.031	2.2±1.6
0.2	Minimal repair	2.552±0.298	0.000	0.000	0.0
0.2	PALR rule	2.652±0.297	0.000	0.200±0.064	0.0
0.8	Greedy global reoptimization	4.664±0.754	0.076±0.037	0.138±0.064	10.2±4.5
0.8	Minimal repair	4.422±0.701	0.000	0.000	0.0
0.8	PALR rule	4.613±0.712	0.000	0.200±0.064	0.0

"value harvesting under stable geometry" from "deep rearrangement" more clearly than the other disturbances.

5.5 Compatibility Study

To evaluate compatibility with stronger search, we combine the PALR repair contract with inference-time PUCT search and learned priors under the same frozen-aware framework. In the heavily frozen urgent regime, the learned-prior PUCT scheme increases urgent utility to 2.474 while matching the maximum observed strict-success rate of 0.34, approximately 4.2% above the 2.374 achieved by greedy global reoptimization. This result shows that external search and learned priors can operate within the same frozen-prefix repair contract in the evaluated setting. The latency results support a hierarchical interpretation: PALR remains sub-millisecond, greedy global reoptimization is several times slower, and learned-prior PUCT is approximately three orders of magnitude slower in the most difficult heavily frozen urgent setting (9.122 ms versus 0.008 ms). These results motivate using bounded local repair as a fast default path and escalating to stronger search when the additional utility justifies the latency cost.

6 Conclusion

We formulate dynamic Earth-observation scheduling as the repair of a partially issued plan and evaluate PALR as a bounded local response. The results support PALR most clearly after cancellation: it matches the utility band of greedy global reoptimization while introducing substantially less disruption. Reward changes reveal a second case in which unchanged geometry allows PALR to obtain value through small-scale tail insertions without reopening retained work. Its boundaries are also clear: urgent insertion degenerates to minimal repair when there is no frozen prefix; at FR = 0.6, stronger search improves value while matching the maximum observed strict-success rate of 17 out of 50, and does so with greater latency and movement. PALR is therefore not a universal replacement for

heavier search, but a credible rapid first response whose escalation conditions can be identified experimentally.

References

[1] Saad Attieh, Nguyen Dang, Christopher Jefferson, Ian Miguel, and Peter Nightingale. 2025. Athanor: Local search over abstract constraint specifications. *Artificial Intelligence* 340 (2025), 104277. doi:10.1016/j.artint.2024.104277

[2] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. 2021. Machine Learning for Combinatorial Optimization: A Methodological Tour d'Horizon. *European Journal of Operational Research* 290, 2 (2021), 405–421. doi:10.1016/j.ejor.2020.07.063

[3] Shaked Brody, Uri Alon, and Eran Yahav. 2021. How Attentive are Graph Attention Networks? arXiv:2105.14491 [cs.LG] https://arxiv.org/abs/2105.14491

[4] Junyang Cai, Taoan Huang, and Bistra Dilkina. 2024. Learning Backdoors for Mixed Integer Linear Programs with Contrastive Learning. In *ECAI 2024*. doi:10.3233/faia240768

[5] Margarida Caleiras, Samuel Moniz, and Paulo Nascimento. 2026. A Constraint Programming Model for the Super-Agile Earth Observation Satellite Imaging Scheduling Problem. In *Proceedings of the 15th International Conference on Operations Research and Enterprise Systems*. 110–121. doi:10.5220/0014407000004055

[6] Quentin Cappart, Didier Chételat, Elias B. Khalil, Andrea Lodi, Christopher Morris, and Petar Velickovic. 2023. Combinatorial Optimization and Reasoning with Graph Neural Networks. *Journal of Machine Learning Research* 24, 130 (2023), 1–61. https://jmlr.org/papers/v24/22-0459.html

[7] Tristan Cazenave. 2021. Batch Monte Carlo Tree Search. arXiv:2104.04278 [cs.AI] https://arxiv.org/abs/2104.04278

[8] Jiawei Chen, Feiran Wang, Yingguo Chen, Lei He, Yonghao Du, Jian Wu, and Yingwu Chen. 2025. A Generalized Bilevel Optimization Model for Large-Scale Task Scheduling in Multiple Agile Earth Observation Satellites. *Knowledge-Based Systems* 309 (2025), 112809. doi:10.1016/j.knosys.2024.112809

[9] Rubing Chen, T. C. E. Cheng, C. T. Ng, Jun-Qiang Wang, Hongjun Wei, and Jinjiang Yuan. 2024. Rescheduling to trade off between global disruption of original jobs with flexibility and scheduling cost of new jobs. *Omega* 128 (2024), 103114. doi:10.1016/j.omega.2024.103114

[10] Weizhe Chen, Sven Koenig, and Bistra Dilkina. 2024. MARL-LNS: Cooperative Multi-agent Reinforcement Learning via Large Neighborhoods Search. arXiv:2404.03101 [cs.MA] https://arxiv.org/abs/2404.03101

[11] Xiaoen Feng, Yuqing Li, and Minqiang Xu. 2024. Multi-satellite cooperative scheduling method for large-scale tasks based on hybrid graph neural network and metaheuristic algorithm. *Advanced Engineering Informatics* 60 (2024), 102362. doi:10.1016/j.aei.2024.102362

[12] Benedetta Ferrari, Jean-François Cordeau, Maxence Delorme, Manuel Iori, and Roberto Orosei. 2025. Satellite Scheduling Problems: A Survey of Applications in Earth and Outer Space Observation. *Computers & Operations Research* 173 (2025), 106875. doi:10.1016/j.cor.2024.106875

[13] Chengrui Gao, Haopu Shang, Ke Xue, Dong Li, and Chao Qian. 2024. Towards Generalizable Neural Solvers for Vehicle Routing Problems via Ensemble with Transferrable Local Policy. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*. 6914–6922. doi:10.24963/ijcai.2024/764

[14] Andoni I. Garmendia, Quentin Cappart, Josu Ceberio, and Alexander Mendiburu. 2024. MARCO: A Memory-Augmented Reinforcement Framework for Combinatorial Optimization. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*. 6931–6939. doi:10.24963/ijcai.2024/766

[15] Gregor Hendel. 2022. Adaptive large neighborhood search for mixed integer programming. *Mathematical Programming Computation* 14, 2 (2022), 185–221. doi:10.1007/s12532-021-00209-7

[16] Sofia Holguin Jimenez, Wajdi Trabelsi, and Christophe Sauvey. 2024. Multi-Objective Production Rescheduling: A Systematic Literature Review. *Mathematics* 12, 20 (2024), 3176. doi:10.3390/math12203176

[17] Andre Hottung and Kevin Tierney. 2022. Neural Large Neighborhood Search for Routing Problems. *Artificial Intelligence* 313 (2022), 103786. doi:10.1016/j.artint.2022.103786

[18] Antoine Jacquet, Guillaume Infantes, Emmanuel Benazera, Vincent Baudoui, Jonathan Guerra, and Stéphanie Roussel. 2024. Earth Observation Satellite Scheduling with Graph Neural Networks and Monte Carlo Tree Search. arXiv:2408.15041 [cs.AI] https://arxiv.org/abs/2408.15041

[19] Shai Krigman, Tal Grinshpoun, and Lihi Dery. 2024. Scheduling of Earth Observing Satellites Using Distributed Constraint Optimization. *Journal of Scheduling* 27 (2024), 507–524. doi:10.1007/s10951-024-00816-x

[20] Stefan Lendl, Ulrich Pferschy, and Elena Rener. 2024. Rescheduling with New Orders Under Bounded Disruption. *INFORMS Journal on Computing* 36, 6 (2024), 1654–1675. doi:10.1287/ijoc.2023.0038

[21] H. Li, Y. Li, Q. Q. Meng, X. Li, L. Shao, and S. Zhao. 2024. An Onboard Periodic Rescheduling Algorithm for Satellite Observation Scheduling Problem with Common Dynamic Tasks. *Advances in Space Research* 73, 11 (2024), 5242–5253. doi:10.1016/j.asr.2024.02.038

[22] Peiyan Li, Peixing Cui, and Huiquan Wang. 2025. Mission Sequence Model and Deep Reinforcement Learning-Based Replanning Method for Multi-Satellite Observation. *Sensors* 25, 6 (2025), 1707. doi:10.3390/s25061707

[23] Peiyan Li, Huiquan Wang, Yongxing Zhang, and Ruixue Pan. 2024. Mission Planning for Distributed Multiple Agile Earth Observing Satellites by Attention-Based Deep Reinforcement Learning Method. *Advances in Space Research* 74, 5 (2024), 2388–2404. doi:10.1016/j.asr.2024.06.003

[24] Zhuoyi Lin, Yaoxin Wu, Bangjian Zhou, Zhiguang Cao, Wen Song, Yingqian Zhang, and Senthilnath Jayavelu. 2024. Cross-Problem Learning for Solving Vehicle Routing Problems. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*. 6958–6966. doi:10.24963/ijcai.2024/769

[25] Z. Liu, W. Xiong, C. Han, and X. Yu. 2024. Deep Reinforcement Learning with Local Attention for Single Agile Optical Satellite Scheduling Problem. *Sensors* 24, 19 (2024), 6396. doi:10.3390/s24196396

[26] Yaosong Long, Chengjun Shan, Wei Shang, Jin Li, and Yulin Wang. 2024. Deep Reinforcement Learning-Based Approach With Varying-Scale Generalization for the Earth Observation Satellite Scheduling Problem Considering Resource Consumptions and Supplements. *IEEE Trans. Aerospace Electron. Systems* 60, 3 (2024), 2572–2585. doi:10.1109/TAES.2024.3379169

[27] Tommaso Mannelli Mazzoli, Lucas Kletzander, Pascal Van Hentenryck, and Nysret Musliu. 2024. Investigating Large Neighbourhood Search for Bus Driver Scheduling. In *Proceedings of the 34th International Conference on Automated Planning and Scheduling (ICAPS)*. 360–368. doi:10.1609/icaps.v34i1.31495

[28] Nada Ouahabi, Ahmed Chebak, Oulaid Kamach, Oussama Laayati, and Mourad Zegrari. 2024. Leveraging digital twin into dynamic production scheduling: A review. *Robotics and Computer-Integrated Manufacturing* 89 (2024), 102778. doi:10.1016/j.rcim.2024.102778

[29] Jing Qi, Min Hu, Lining Xing, and Zhenhua Su. 2025. Deep Reinforcement Learning-Assisted Large Neighborhood Search for Online Scheduling Large-Scale Emergency Tasks to Earth-Observing Satellites. *Expert Systems with Applications* 291 (2025), 128502. doi:10.1016/j.eswa.2025.128502

[30] Robbert Reijnen, Yingqian Zhang, Hoong Chuin Lau, and Zaharah Bukhsh. 2022. Online Control of Adaptive Large Neighborhood Search using Deep Reinforcement Learning. arXiv:2211.00759 [cs.LG] https://arxiv.org/abs/2211.00759

[31] Nicolas Sonnerat, Pengming Wang, Ira Ktena, Sergey Bartunov, and Vinod Nair. 2021. Learning a Large Neighborhood Search Algorithm for Mixed Integer Programs. arXiv:2107.10201 [math.OC] https://arxiv.org/abs/2107.10201

[32] Zhiqing Sun and Yiming Yang. 2023. DIFUSCO: Graph-Based Diffusion Solvers for Combinatorial Optimization. In *Advances in Neural Information Processing Systems 36*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/0ba520d93c3df592c83a611961314c98-Abstract-Conference.html

[33] Xin Wang, Fanyu Zhao, Zhong Shi, and Zhonghe Jin. 2023. Deep Reinforcement Learning-Based Periodic Earth Observation Scheduling for Agile Satellite Constellation. *Journal of Aerospace Information Systems* 20, 8 (2023), 508–519. doi:10.2514/1.I011209

[34] Tao Wu, Huachao Cui, Tao Guan, Yuesong Wang, and Yan Jin. 2024. ReinforceNS: Reinforcement Learning-Based Multi-Start Neighborhood Search for Solving the Traveling Thief Problem. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*. 7038–7046. doi:10.24963/ijcai.2024/778

[35] Yaoxin Wu, Wen Song, Zhiguang Cao, and Jie Zhang. 2021. Learning Large Neighborhood Search Policy for Integer Programming. arXiv:2111.03466 [cs.AI] https://arxiv.org/abs/2111.03466

[36] Haoran Ye, Jiarui Wang, Helan Liang, Zhiguang Cao, Yong Li, and Fanzhang Li. 2024. GLOP: Learning Global Partition and Local Construction for Solving Large-Scale Routing Problems in Real-Time. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 20284–20292. doi:10.1609/aaai.v38i18.30009

[37] Chao Zhang, Max Juraschek, and Christoph Herrmann. 2024. Deep reinforcement learning-based dynamic scheduling for resilient and sustainable manufacturing: A systematic review. *Journal of Manufacturing Systems* 77 (2024), 962–989. doi:10.1016/j.jmsy.2024.10.026

[38] Junjie Zhang, Canhui Luo, Zhouxing Su, Qingyun Zhang, Zhipeng Lu, Junwen Ding, and Yan Jin. 2025. NS4S: Neighborhood Search for Scheduling Problems Via Large Language Models. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI)*. 8687–8695. doi:10.24963/ijcai.2025/966

[39] Jianan Zhou, Yaoxin Wu, Zhiguang Cao, Wen Song, Jie Zhang, and Zhenghua Chen. 2023. Learning Large Neighborhood Search for Vehicle Routing in Airport Ground Handling. arXiv:2302.13797 [cs.AI] https://arxiv.org/abs/2302.13797